# Wireless SNMP Log Analysis

## A Cisco WLC Security Operations Dashboard

**Project Type:** CSIS Post-Graduate (PG) Project

**Domain:** Network Security / WLAN Operations

**Technology Stack:** Python · FastAPI · Vanilla JS · Chart.js

**Date:** 13 May 2026

*"A full-stack tool that ingests Cisco Wireless LAN Controller (WLC) SNMP trap logs, performs analysis across clients, access points and SSIDs, and presents the findings in an interactive security operations dashboard."*


**Team Members**

**Lakshay Baijal** Roll No. 2024202006

**Shreya Koka** Roll No. 2024202004

**S Supraja** Roll No. 2024202014

**Supervisor:** Prof. Shatrunjay Rawat


**GitHub Repository:**

https://github.com/LakshayBaijal/PG-Project-Log-Analysis

# Contents

# 1 Project Overview

The **PG Project – Wireless SNMP Log Analysis** is an end-to-end network security analytics system purpose-built for Cisco Wireless LAN Controller environments. The core problem it solves is straightforward: enterprise wireless networks generate thousands of SNMP trap messages per hour, each containing rich information about client associations, authentication events, de-authentications, roaming transitions, and rogue device detections. Manually reading these logs to detect anomalies is infeasible at scale. The analysis and images included in the report are after processing SNMP logs of IIIT Hyderabad for the day, 2026-05-10 (10th May 2026)

This project automates the entire pipeline:

1. A raw `.log` file (Zabbix SNMP trap format) is uploaded through a web interface or a pre-loaded sample is selected.
2. A Python backend parses every trap block, resolves OIDs against a comprehensive MIB database, and runs twelve categories of behavioural analysis.
3. A JSON report is returned to the frontend and rendered as an interactive, chart-driven security dashboard with six navigable views.
4. Reports are also persisted locally (JSON + raw log) for audit purposes.

**Target audience**

Network operations teams and security analysts responsible for enterprise WiFi infrastructure running Cisco WLC/Catalyst Center. The tool requires no SNMP expertise from the analyst – all OID resolution is handled internally.

# 2 System Architecture

## 2.1 High-Level Design

The system follows a clean client-server architecture with a Python backend and a browser-based single-page application (SPA) frontend.

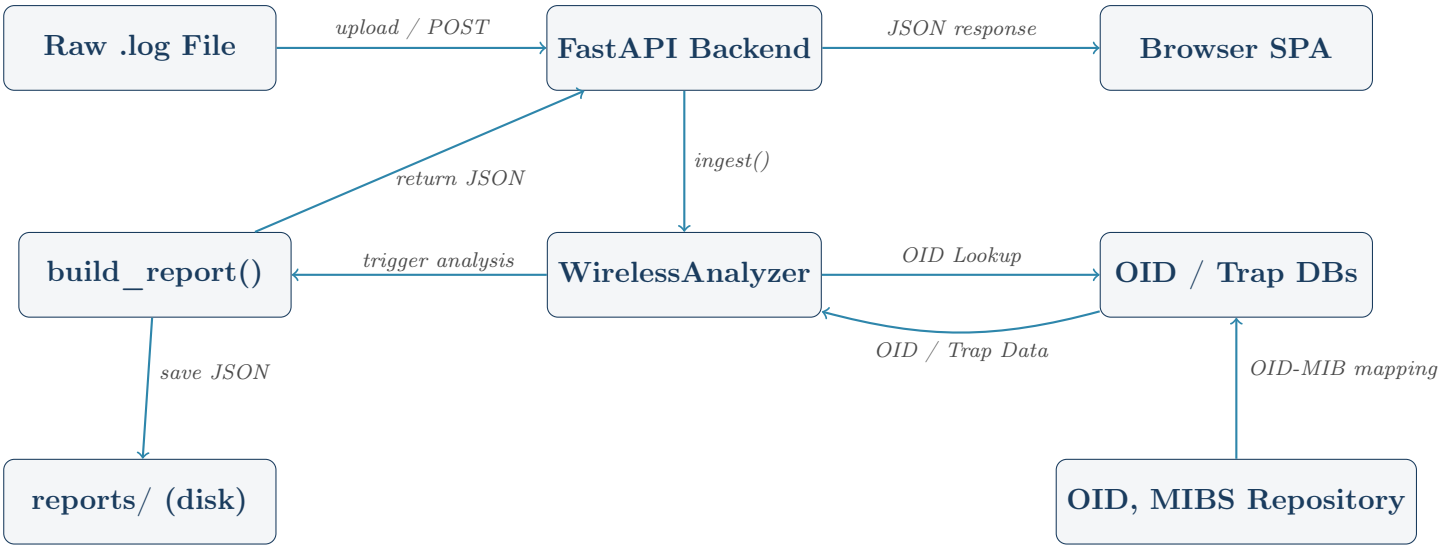


Figure 1: High-level system architecture

## 2.2 Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| **Web Server** | Uvicorn (ASGI) | High-performance async server |
| **API** | FastAPI | REST endpoints, file upload, CORS |
| **Analysis** | Pure Python 3.10 | Parsing, statistics, report building |
| **OID Database** | JSON trie + flat map | Fast MIB lookup without SNMP tools |
| **Frontend** | Vanilla JS + CSS | Single-page dashboard |
| **Charts** | Chart.js | Interactive bar, line, and doughnut charts |
| **Icons** | Font Awesome 6 | UI icon set |
| **Fonts** | Google Fonts (Inter) | Typography |

Table 1: Technology stack summary

## 2.3 Directory Structure

```
PG-Project-Log-Analysis/
  main.py                  # FastAPI application entry point
  wireless_analyzer.py     # Core analysis engine (1452 lines)
  split_log.py             # Utility: split large log files
     into chunks
  test_counts.py           # Utility: quick event-type count
     check
  requirements.txt         # fastapi, uvicorn, python-multipart
     , aiofiles
  snmptrap-20250522.log    # Live sample log (156 MB)
```

```
  snmptrap-sample.log       # Short demo log
  oid_trie.json             # Trie-indexed MIB OID database
  oid_flat.json             # Flat OID lookup table
  name_to_oid.json          # Reverse OID database (name -> OID)
  trap_db.json              # Trap-specific OID metadata
  result.json               # Cached sample analysis result
  frontend/
    index.html              # SPA shell with six navigation
       sections
    app.js                  # Dashboard rendering + chart logic
       (~29 KB)
    style.css               # Custom CSS (sidebar, cards, charts
       )
  reports/                  # Auto-created; stores timestamped
     JSON + logs
  backup/
    analyzer.py             # Earlier prototype of the analyzer
```

Listing 1: Repository layout (key files)

# 3 Backend: Log Parsing and Analysis Engine

The heart of the project is `wireless_analyzer.py`, a 1,452-line Python module that implements the full analysis pipeline from raw log text to structured JSON report.

## 3.1 SNMP Trap Log Format

The input logs follow the Zabbix SNMP trap receiver format. Each trap block begins with a timestamp and source IP, followed by PDU information and a `VARBINDS` section containing OID-value pairs.

```
2026-03-20T00:00:02+0530 ZBXTRAP 10.4.5.75
PDU INFO:
  receivedfrom    UDP: [10.4.5.75]:32768->[10.4.31.226]:162
  notificationtype TRAP
VARBINDS:
  iso.3.6.1.2.1.1.3.0              type=67 value=Timeticks:
     (703050804) ...
  iso.3.6.1.6.3.1.1.4.1.0         type=6  value=OID: iso
     .3.6.1.4.1.14179.2.6.3.2
```

```
iso.3.6.1.4.1.14179.2.6.2.35.0 type=4  value=Hex-STRING: 00 1
   D 46 7C 87 80
iso.3.6.1.4.1.14179.2.2.1.1.3.0 type=4 value=STRING: "
   AP_Name_Classroom"
```

Listing 2: Example trap block (association event)

## 3.2 OID Resolution Strategy

One of the most technically demanding aspects of the project is resolving vendor-specific Cisco OIDs to semantic field names without any external SNMP library dependency. A three-tier resolution chain is used:

1. **Trie lookup** – The OID is traversed through `oid_trie.json`, a trie-indexed structure that allows efficient prefix matching and returns MIB metadata including the human-readable field name.
2. **Inference from field name** – The resolved name is pattern-matched (e.g., *apname*, *ssid*, *rssi*, *bssid*) to classify the value into a canonical field such as `ap_name`, `client_mac`, or `reason_code`.
3. **Static OID map fallback** – A hard-coded dictionary (`OID_FIELD_MAP`) covers the most common Cisco WLC OIDs from the `.9.9.599` and AireSpace `.14179` branches.

Additionally, a reverse MAC address extraction technique reads the OID suffix (e.g., `.27.14.145.151.98.238.223`) and converts the dotted decimal encoding to a colon-separated MAC where appropriate.

## 3.3 Parsing Pipeline

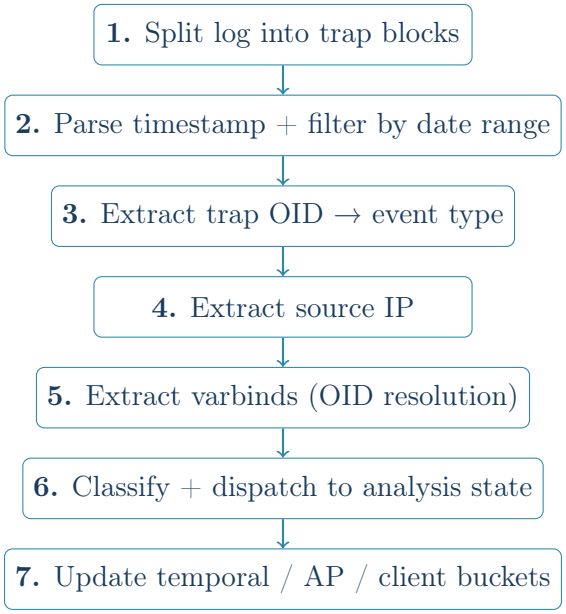


Figure 2: Per-trap-block parsing pipeline inside `WirelessAnalyzer.ingest()`

## 3.4 Event Classification

Trap OIDs are mapped to semantic event types using two sources:

| OID (abbreviated) | Event Type | Source MIB |
|---|---|---|
| `.9.9.599.0.1` | `assoc` | Cisco WLC SNMP MIB |
| `.9.9.599.0.2` | `reassoc` | Cisco WLC SNMP MIB |
| `.9.9.599.0.3` | `disassoc` | Cisco WLC SNMP MIB |
| `.9.9.599.0.4` | `deauth` | Cisco WLC SNMP MIB |
| `.9.9.599.0.8` | `auth_fail` | Cisco WLC SNMP MIB |
| `.9.9.599.0.10` | `roam` | Cisco WLC SNMP MIB |
| `.9.9.599.0.30` | `rogue_ap_detected` | Cisco WLC SNMP MIB |
| `.14179.2.6.3.2` | `assoc` | AireSpace MIB |
| `.14179.2.6.3.53` | `deauth` | AireSpace MIB |
| `.6.3.1.1.5.3` | `link_down` | SNMPv2-MIB |

Table 2: Key trap OID to event type mappings

## 3.5 Session Pairing and State Tracking

The `WirelessAnalyzer` class maintains extensive in-memory state to correlate individual events into sessions and detect patterns:

- **Association pairing:** A `_pending_assoc` dictionary holds the most recent `assoc`/`reassoc` event per MAC address. When a subsequent `deauth` or `disassoc` is observed for the same MAC, the pair is resolved into a complete session with a start time, end time, duration, AP, SSID, and disconnect reason.
- **Orphan handling:** Disconnect events with no matching start (log truncation, log rotation) are recorded as zero-duration sessions rather than discarded, preserving visibility.
- **Roaming detection:** An `assoc` event whose AP differs from the previous `assoc` in `_pending_assoc` is classified as a roam transition, building a per-client roam path.
- **Correlated deauth/auth-fail deduplication:** Events within a 2-second window are cross-checked to avoid double-counting a single failure as both a deauth and an auth-fail.
- **Presence sessions:** A separate 5-minute sliding window (`session_timeout = 300 s`) tracks how long each user/MAC identity was active on each AP regardless of whether the underlying SNMP session was cleanly paired.

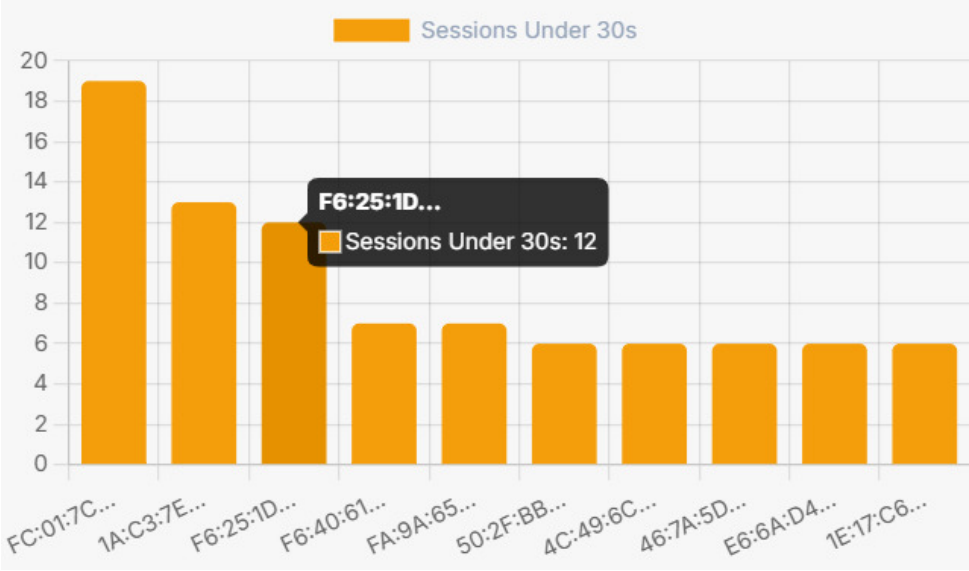


Figure 3: Dashboard view: top client MACs ranked by number of sessions lasting under 30 seconds, highlighting devices with frequent short-lived or unstable associations.

## 3.6 Deauth Storm Detection

De-authentication storms – a common indicator of denial-of-service activity or misconfiguration – are detected using a sliding-window algorithm:

```python
for ap, times in deauth_times_by_ap.items():
    times.sort()
    i = 0
    while i < len(times):
        t = times[i]
        window = [x for x in times[i:] if (x - t).total_seconds
            () <= 60]
        if len(window) >= 3:
            deauth_windows.append({
                "ap": ap, "window_start": t,
                "window_end": window[-1],
                "count": len(window),
                "duration_sec": (window[-1] - t).total_seconds
                    (),
            })
            i += len(window)    # advance past this window (no
                overlap)
        else:
            i += 1
```

Listing 3: Deauth burst window algorithm

Any access point experiencing three or more de-authentications within a 60-second window is flagged with the exact window boundaries and event count.

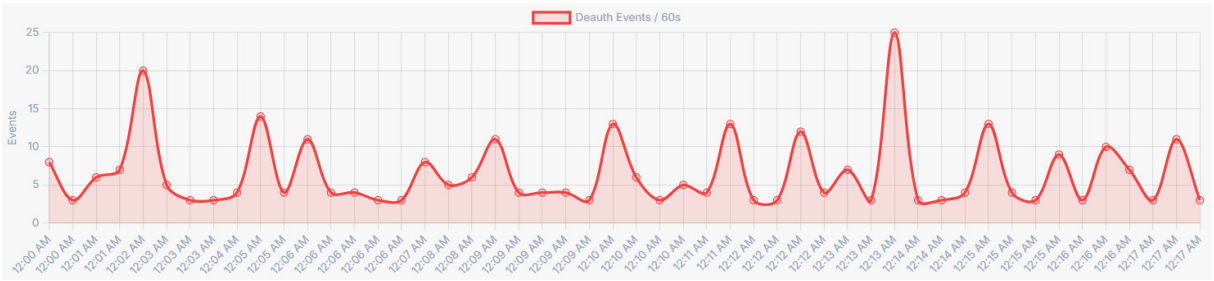


Figure 4: Deauth events per 60-second window over an 18-minute observation period. Burst peaks (up to 25 events/min) correspond to windows flagged by the sliding-window storm detection algorithm.

## 3.7 Report Sections Produced

The `build_report()` function assembles a comprehensive JSON document with twelve top-level sections:

| **Section** | Contents |
|---|---|
| **overview** | Total trap count, unique MACs, APs, SSIDs, auth failure counts, roam totals, rogue events, deauth windows |
| **ap_summary** | Per-AP: unique clients, one-time vs repeat visitors, sessions, auth failures, deauths, disassocs, peak hour, link flaps |
| **clients** | Per-client: first/last seen, all APs visited, SSIDs used, roam path, session stats, off-hours sessions, gap analysis, reason code breakdown |
| **auth_failure_analysis** | Failure counts per client, per AP, per SSID; clients that failed and never successfully associated |
| **deauth_windows** | Every 60-second deauth burst window, sorted by count |
| **roaming_analysis** | Total transitions, top roaming clients, most common AP-to-AP transition pairs |
| **ssid_analysis** | Per-SSID client count, AP coverage, sessions, failures, single-AP flag |
| **temporal_analysis** | Events by hour of day, events by day of week, top 5-minute burst windows |
| **off_hours_activity** | Clients active outside 08:00–20:00, with event types and active hours |
| **rogue_events** | All rogue AP/client detection events with raw fields |
| **ap_link_flaps** | APs with link-up/link-down events and their timestamps |
| **multi_ssid_clients** | Clients that connected to more than one SSID |
| **unclosed_sessions** | Clients with an open association and no disconnect seen |
| **ap_client_sessions** | Presence sessions per AP (identity, start, end, duration, SSIDs) |

Table 3: Report sections generated by `build_report()`

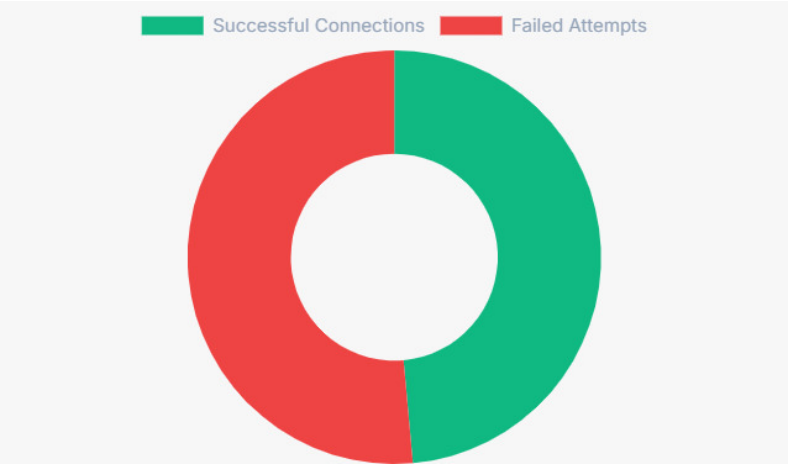


Figure 5: Overall authentication outcome ratio: successful connections vs failed attempts. The near-even split indicates a significant authentication failure rate warranting investigation under the `auth_failure_analysis` report section.

# 4 API Layer

The FastAPI application in `main.py` exposes a minimal but complete REST interface.

## 4.1 Endpoints

| Method | Path | Description |
|---|---|---|
| `POST` | `/api/analyze` | Accepts a multipart form with the log file (`UploadFile`) and optional `start_datetime` / `end_datetime` ISO-8601 strings. Runs the full analysis and returns the JSON report. |
| `GET` | `/api/sample` | Runs the analysis on the bundled `snmptrap-20250522.log` sample file with optional date range filtering. Useful for demoing without an upload. |
| `GET` | `/{path}` | Serves the static SPA from the `frontend/` directory (mounted via `StaticFiles`). |

Table 4: API endpoints

## 4.2 Report Persistence

Every analysis run saves two files under `reports/` with a timestamp:

- `wireless_report_YYYYMMDD_HHMMSS.json` – the complete structured report.
- `wireless_log_YYYYMMDD_HHMMSS.log` – the raw uploaded log, preserved for re-analysis or audit.

### 4.3 Date Range Filtering

Both endpoints accept optional `start_datetime` and `end_datetime` parameters. When supplied, the parser silently skips any trap block whose timestamp falls outside the window, enabling targeted forensic analysis of a specific incident time window without modifying the source log.

## 5 Frontend: Security Operations Dashboard

### 5.1 Application Structure

The frontend is a vanilla JavaScript SPA with no build step or framework dependency. Chart.js is loaded from CDN for data visualisation. The sidebar navigation scrolls smoothly between six content sections rendered from the API response.

### 5.2 Navigation Sections

| Nav Item | Content |
|---|---|
| Dashboard | Overview KPI cards + key charts |
| Threat Intel | Auth failures, deauth windows, rogue events, off-hours activity |
| Network Health | AP link flaps, deauth storm timeline, session health |
| Client Activity | Per-client drill-down table with sessions, roam paths |
| AP Analytics | Per-AP stats, peak hour heat map, top APs by traffic |
| AP Client Sessions | Presence sessions per AP in tabular form |

Table 5: Dashboard navigation sections

### 5.3 Charts and Visualisations

The dashboard renders up to nine Chart.js instances, each destroyed and re-created on each new analysis to avoid canvas conflicts:

- **AP Client Bar Chart** – Unique clients per access point.

- **Auth Failure Chart** – Top clients by authentication failure count.
- **SSID Doughnut** – Client distribution across SSIDs.
- **Disconnect Reasons Chart** – Frequency of 802.11 reason codes.
- **Active Users Timeline** – Events-per-hour line chart.
- **Deauth Timeline** – 5-minute burst window counts over time.
- **Unstable Clients Chart** – Clients ranked by off-hours sessions.
- **Activity Heatmap** – Events by hour and weekday.
- **Session Duration Histogram** – Distribution of session lengths.

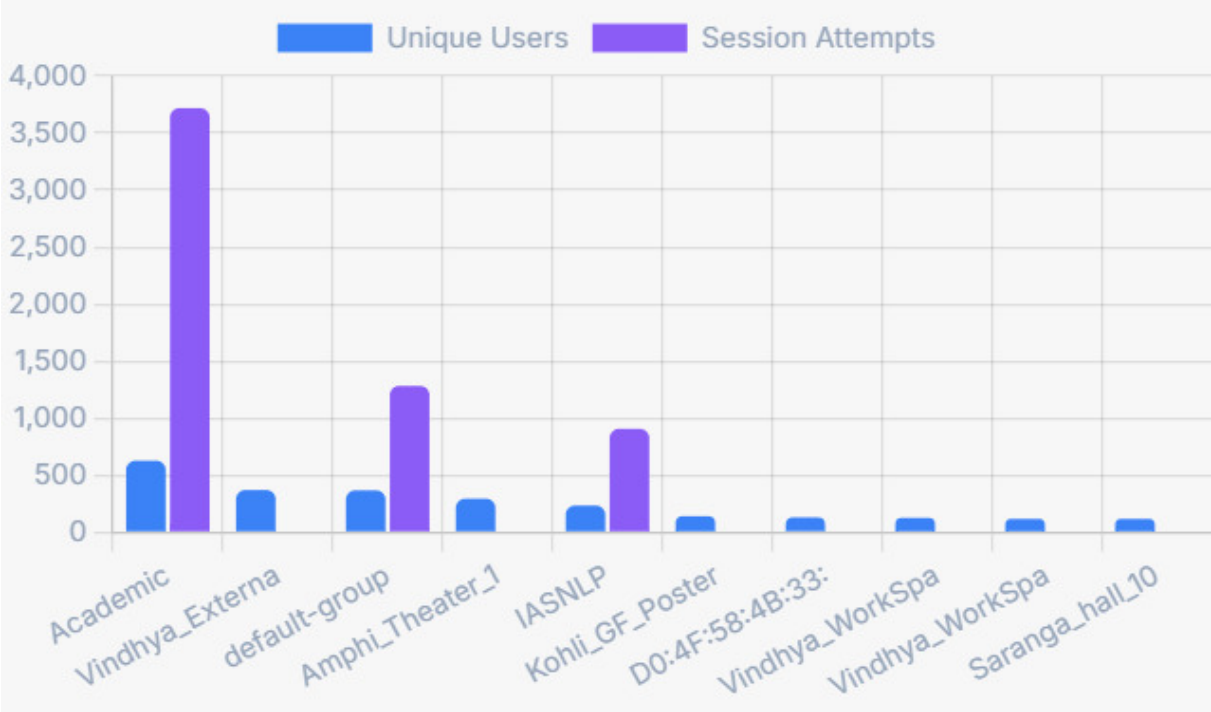


Figure 6: AP Client Bar Chart: unique users (blue) and total session attempts (purple) per access point / SSID group. The *Academic* AP dominates with over 600 unique users and ∼3,700 session attempts, followed by *default-group* and *Amphi_ Theater_ 1.*

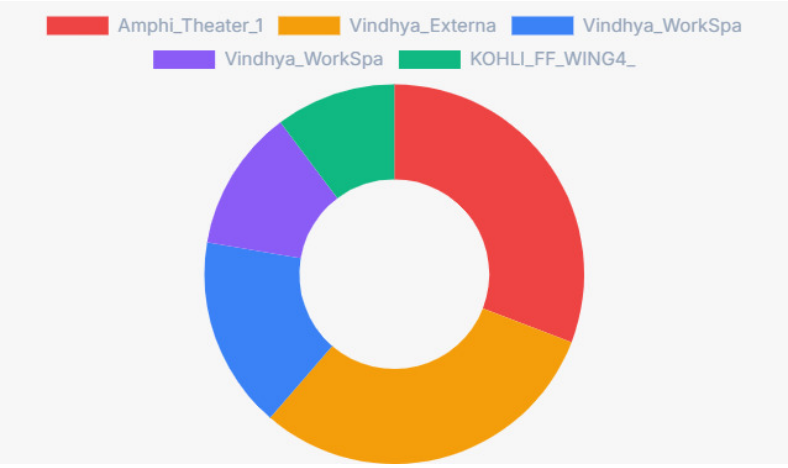


Figure 7: SSID Doughnut chart: client session distribution across the five most-used SSIDs. *Amphi_Theater_1* and *Vindhya_Externa* account for the majority of sessions, with *KOHLI_FF_WING4* representing a smaller dedicated segment.

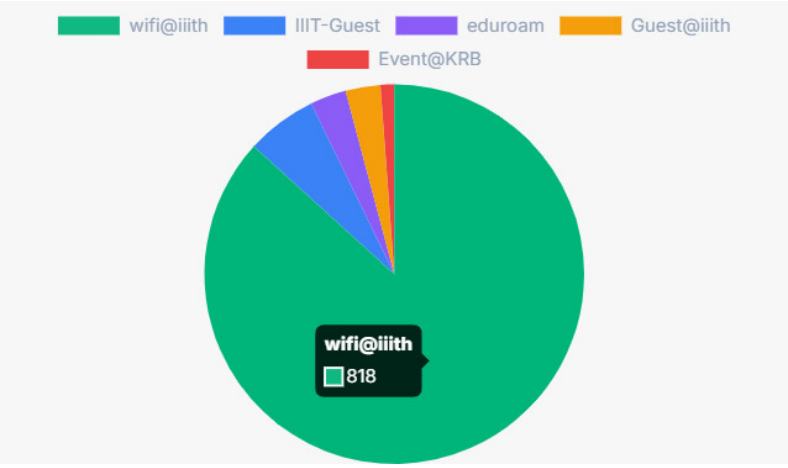


Figure 8: Unique-client distribution by SSID. `wifi@iiith` dominates with 818 unique clients – consistent with its role as the primary campus SSID at IIIT Hyderabad – while *IIIT-Guest*, *eduroam*, *Guest@iiith*, and *Event@KRB* serve progressively smaller populations.

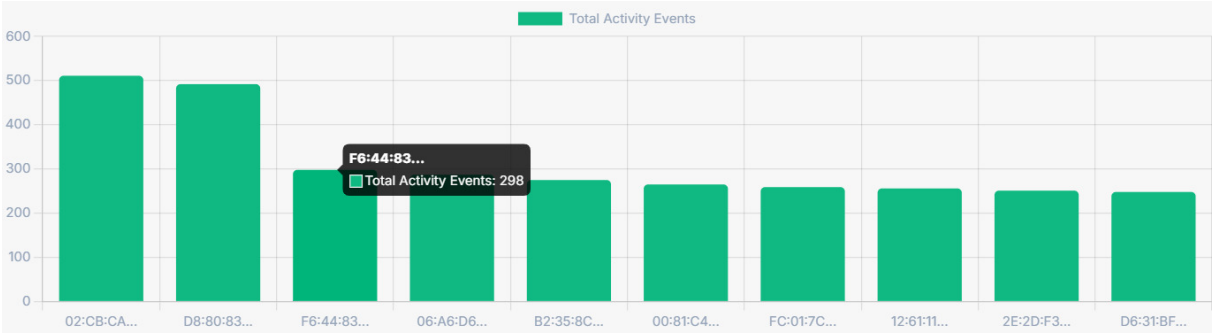


Figure 9: Top 10 client MACs ranked by total activity events. The two highest-activity devices (MACs `02:CB:CA...` and `D8:80:83...`) each generated over 480 events, substantially above the 240–300 range of the remaining eight, suggesting potential infrastructure devices or highly mobile users.

### 5.4 Upload and Sample Workflow

The sidebar provides two entry paths:

1. **Upload Log** – The analyst selects a `.log` or `.txt` file, optionally applies a date range, and clicks *Analyze File*. The file is sent as a `multipart/form-data` POST to `/api/analyze`.
2. **Run Sample Data** – Triggers a GET to `/api/sample`, which runs the bundled production log through the same pipeline. This path allows instant demonstration without requiring an actual log file.

## 6 Utility Scripts

### 6.1 split_log.py

Splits the production log (156 MB) into configurable chunks (default 30 MB) for incremental analysis or upload to size-limited systems. Operates line-by-line in streaming mode to avoid loading the entire file into memory.

### 6.2 test_counts.py

A quick diagnostic script that reads the first 20 MB of a log, splits it into trap blocks, and prints a frequency count of event types (assoc, deauth, auth_fail, etc.). Used during development to verify that OID mappings were correctly resolving the dominant trap types in the live log.

# 7 Major Learnings

This section documents the most significant technical and conceptual insights gained during the project.

## 7.1 SNMP OID Resolution Without External Libraries

A foundational learning was the feasibility of building a complete, dependency-free OID resolution engine. Instead of relying on `pysnmp` or Net-SNMP, the project pre-processes MIB data into three JSON structures: a trie for efficient prefix matching, a flat dictionary for direct lookup, and a reverse name-to-OID map. This approach keeps the deployment dependency footprint minimal (only FastAPI and uvicorn are needed) while still resolving thousands of enterprise OIDs from both the standard SNMPv2 MIB and Cisco vendor-specific branches.

## 7.2 Trie Data Structures for OID Matching

OIDs are naturally tree-structured (e.g., `.1.3.6.1.4.1.9.9.599.0.4` represents a path through the OID tree). The trie structure mirrors this hierarchy: each node is a dictionary indexed by the next OID component, with a `_value` key storing the MIB metadata at leaf nodes. Traversal is O(d) where d is the OID depth, making lookups faster than regex scanning and naturally handling prefix matching for table-indexed OIDs.

## 7.3 Stateful Session Reconstruction from Stateless Events

SNMP traps are stateless – each is a fire-and-forget notification with no built-in concept of a "session". Reconstructing meaningful sessions from these events required maintaining a per-MAC pending-association dictionary and designing careful rules for handling the many edge cases: orphan disconnects (no matching assoc), simultaneous events at the same second, log file boundaries that cut a session in half, and events from clients that roam mid-session.

## 7.4 Importance of Event Correlation for Accuracy

The project revealed that naively counting deauth and auth-fail events leads to significant over-counting. When an 802.1X authentication times out, the WLC sends both an `auth_fail` trap and an immediate `deauth` trap within milliseconds. Without the 2-second

correlation window that deduplicates these pairs, a single failed authentication attempt would be counted as two separate adverse events, inflating threat metrics.

### 7.5 Python Standard Library is Sufficient for Log-Scale Analytics

All temporal analysis, session pairing, gap analysis, and burst window detection were implemented using only Python built-ins (`defaultdict`, `Counter`, `datetime`, `re`). No pandas, NumPy, or databases were needed for the 156 MB, multi-million-event log. This was an important insight about the efficiency of well-structured in-memory algorithms for this class of problem.

### 7.6 802.11 Protocol Behaviour Visible in Trap Data

Analysing the actual log data provided a direct window into 802.11 state machine behaviour in a production network: the frequency of reason code 4 (inactivity timeout), the prevalence of multi-SSID clients (devices probing multiple networks), the bursty nature of roaming events during peak hours, and the rate at which off-hours events cluster around specific APs rather than being uniformly distributed.

### 7.7 Frontend State Management Without a Framework

Building the dashboard in vanilla JavaScript without React or Vue required careful management of Chart.js instance lifecycle. Each analysis result replaces the previous one, and canvas elements cannot be reused without explicitly destroying the previous Chart instance. Storing all nine chart references in module-level variables and calling `.destroy()` before re-rendering was a straightforward but non-obvious requirement that only became apparent during testing.

## 8 Challenges Faced

## 8.1 Cisco OID Ambiguity Across MIB Versions

**Challenge: OID ambiguity**

The same semantic concept (e.g., client MAC address) appears under different OIDs in the Cisco WLC SNMP MIB (`.9.9.599`) and the older AireSpace MIB (`.14179`). Additionally, OID suffixes often encode table indices that include MAC addresses in dotted-decimal format (e.g., `.14.145.151.98.238.223`), requiring a secondary extraction step on the OID itself, not just its value.

The resolution required building both the trie-based lookup and the `infer_field()` heuristic function, which inspects the resolved MIB field name (e.g., *cldcClientMacAddress*) for substrings like *mac*, *client*, or *ap* to determine the canonical field. Getting the priority order right – AP name before AP MAC before client MAC – took several iterations to prevent mis-classification.

## 8.2 MAC Address Extraction from OID Suffixes

Table-indexed OIDs in SNMP encode the row key in the OID suffix. For client-indexed tables this means a 6-byte MAC in dotted decimal (6 integers) appended to the base OID. Extracting these reliably required a regex that anchored to known table prefix numbers (specifically `.27.` and `.1.`) while excluding known spurious matches (multicast MAC prefixes used as placeholders, e.g., `01:02:01:01:01:00`).

## 8.3 Large Log File Performance

The production log file is approximately 156 MB and contains several million trap blocks. Processing it in a synchronous FastAPI endpoint on the main thread blocks the event loop for the entire analysis duration (tens of seconds). While this was acceptable for a PG project prototype, the architecture does not scale to concurrent users. An `asyncio.run_in_executor` or background task approach would be required for a production deployment.

## 8.4 Timestamp Format Heterogeneity

The log files contained at least three timestamp formats: ISO-8601 with timezone offset (`2026-03-20T00:00:02+0530`), ISO-8601 without timezone, and syslog-style month-day-time strings. A multi-pattern fallback parser with five `strptime` format strings was necessary to handle all variants robustly. The lack of a timezone-aware datetime throughout

the codebase (all datetimes are converted to naive local time) was a pragmatic choice that could cause issues when logs span DST transitions.

### 8.5 Hex-STRING SSID Decoding

SSIDs containing non-ASCII characters are transmitted as space-separated hex bytes (e.g., `48 6F 6D 65`). The `decode_hex_ssid()` function detects this pattern with a regex and decodes it as UTF-8, but SSIDs using other encodings (e.g., legacy encodings) may be displayed as replacement characters. Building a fully robust SSID decoder would require knowledge of the AP's configured character encoding.

### 8.6 Correlated Event Deduplication Ordering

The deauth/auth-fail correlation logic is order-dependent: the algorithm checks recent events in the opposite order depending on which event arrives first. If the `deauth` arrives before the `auth_fail` (the more common case), the deauth is counted initially and then retroactively removed when the auth-fail arrives within 2 seconds. This retroactive removal required popping items from lists, which could produce incorrect results if the timing threshold was exceeded by the log ingestion latency. Extensive testing with the sample log was needed to validate the threshold choice.

### 8.7 Defining Business Hours for a University Network

The off-hours detection uses a hard-coded 08:00–20:00 window. For a university campus (the apparent deployment site, given AP names like *Vindhya_B4-301_Class_Room* and *KOHLI_FF_WING4*), this window is reasonable on weekdays but does not account for weekend schedules, exam periods, or hostel areas where 24-hour activity is normal. A more sophisticated approach would allow per-AP or per-SSID business hour configuration.

## 9 Conclusion and Future Work

The PG Project – Wireless SNMP Log Analysis successfully demonstrates that a lightweight, dependency-minimal Python application can deliver production-grade wireless network security intelligence from raw SNMP trap logs. The twelve-section structured report and interactive dashboard provide security operations teams with the same inferences they would previously derive through hours of manual log analysis, condensed into seconds.

## 9.1 Possible Extensions

- **Async processing:** Move `WirelessAnalyzer.ingest()` to a background task with WebSocket or polling-based progress updates to support concurrent users.
- **Streaming ingestion:** Accept log files via a streaming POST to avoid loading the entire 156 MB into memory before analysis begins.
- **Anomaly scoring:** Add a configurable scoring model that ranks clients by weighted risk (auth failure rate, off-hours activity, short session count, multi-SSID usage) to prioritise analyst attention.
- **Alert integration:** Push high-severity findings (deauth storms, rogue APs, clients with 100% auth failure rate) to a SIEM or ticketing system via webhook.
- **Historical comparison:** Retain analysis results in a database and present trend charts comparing the current report against previous time windows.
- **Configurable business hours:** Allow per-AP or per-SSID business hour definitions through the UI or a JSON configuration file.

**Summary**

This project provided hands-on experience with SNMP trap internals, Cisco WLC MIB structures, stateful event correlation algorithms, trie-based data structures, FastAPI, and single-page application development – all while producing a genuinely useful operational security tool for enterprise wireless environments.